

Лабораторная работа №5

Аппарат сетей Петри

Разанацуа Сара Естэлл

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Обедающие философы	7
3.2	Базовый эксперимент	8
3.3	Анимация процесса	12
3.4	Итоговый отчёт	13
4	Выводы	16
	Список литературы	17

Список иллюстраций

3.1	Рис		7
3.2	Рис		8
3.3	Рис		8
3.4	Рис		9
3.5	Рис		10
3.6	Рис		11
3.7	Рис		12
3.8	Рис		13
3.9	Рис		14
3.10	Рис		15

Список таблиц

1 Цель работы

- Построим сеть Петри для пяти философов, моделируя захват и освобождение вилок. Обнаружим состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую. Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock. Модифицировать сеть, чтобы предотвратить deadlock. Проанализировать результаты и оформить отчёт с графиками и анимацией.

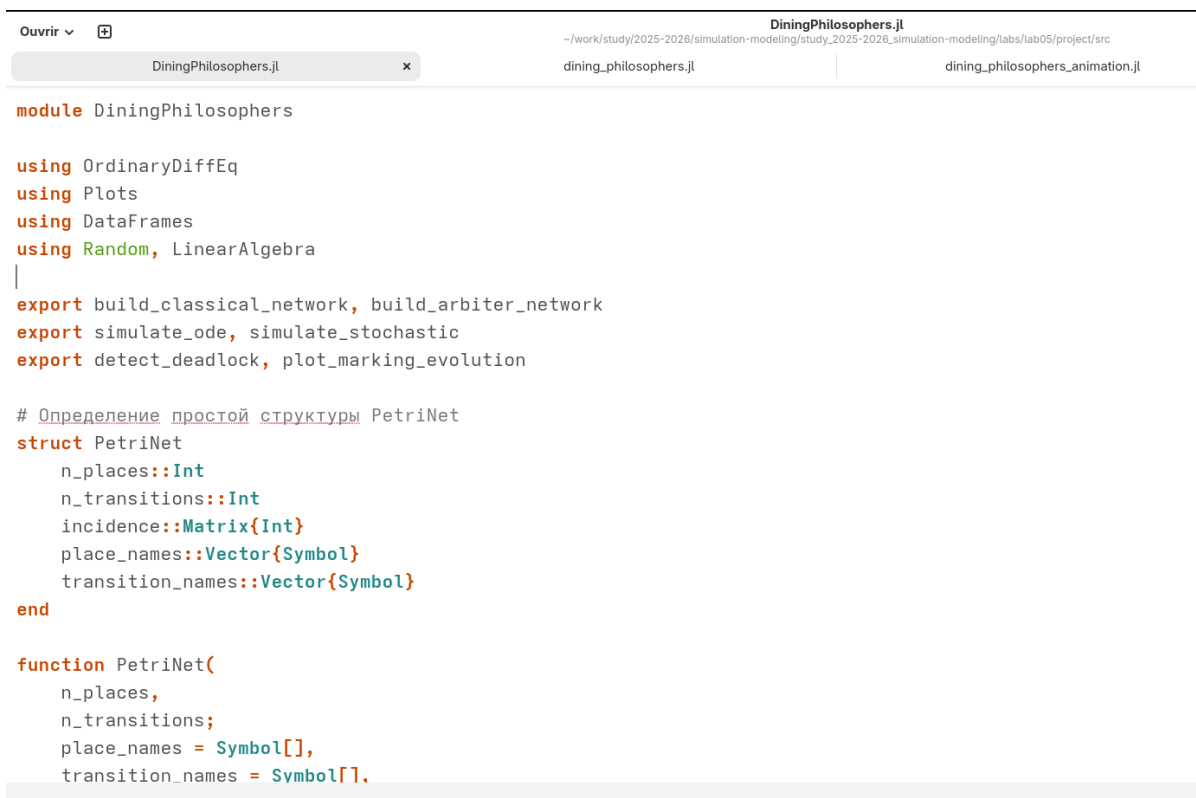
2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
10. Выполнить код из jupyter notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.
 - Результирующие файлы не удаляйте, выложите на git.

3 Выполнение лабораторной работы

3.1 Обедаящие философы

- Мы создадим файл `src/DiningPhilosophers.jl` и запустим его. (рис. 3.1).



The screenshot shows a Jupyter Notebook interface with a single file named `DiningPhilosophers.jl` open. The file path is `~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project/src`. The code in the file is as follows:

```
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
```

Рис. 3.1: Рис

- Вывод в консоль. (рис. 3.2).

```
julia> touch("src/DiningPhilosophers.jl")
"src/DiningPhilosophers.jl"

julia> include("src/DiningPhilosophers.jl")
Main.DiningPhilosophers
```

Рис. 3.2: Рис

3.2 Базовый эксперимент

- Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:

1. классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
2. модифицированная модель с арбитром, которая должна предотвращать deadlock.

(рис. 3.3).

```
using DrWatson
@quickactivate "project"
include(srcdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classic_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)

println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotdir("classic_simulation.png"))
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotdir("arbiter_simulation.png"))
```

Рис. 3.3: Рис

- Создание файла scripts/dining_philosophers.jl и запустим его. (рис. 3.4).


```
julia> include("scripts/dining_philosophers.jl")
Activating project at "~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project"
WARNING: replacing module DiningPhilosophers.
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/serazanacua/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

Рис. 3.4: Рис

- Графики эволюции маркировки позволяют визуально увидеть, как меняются состояния.
1. В классической сети рано или поздно все Eat_i становятся нулевыми, а $Hungry_i$ – единичными (или наоборот).(рис. 3.5).

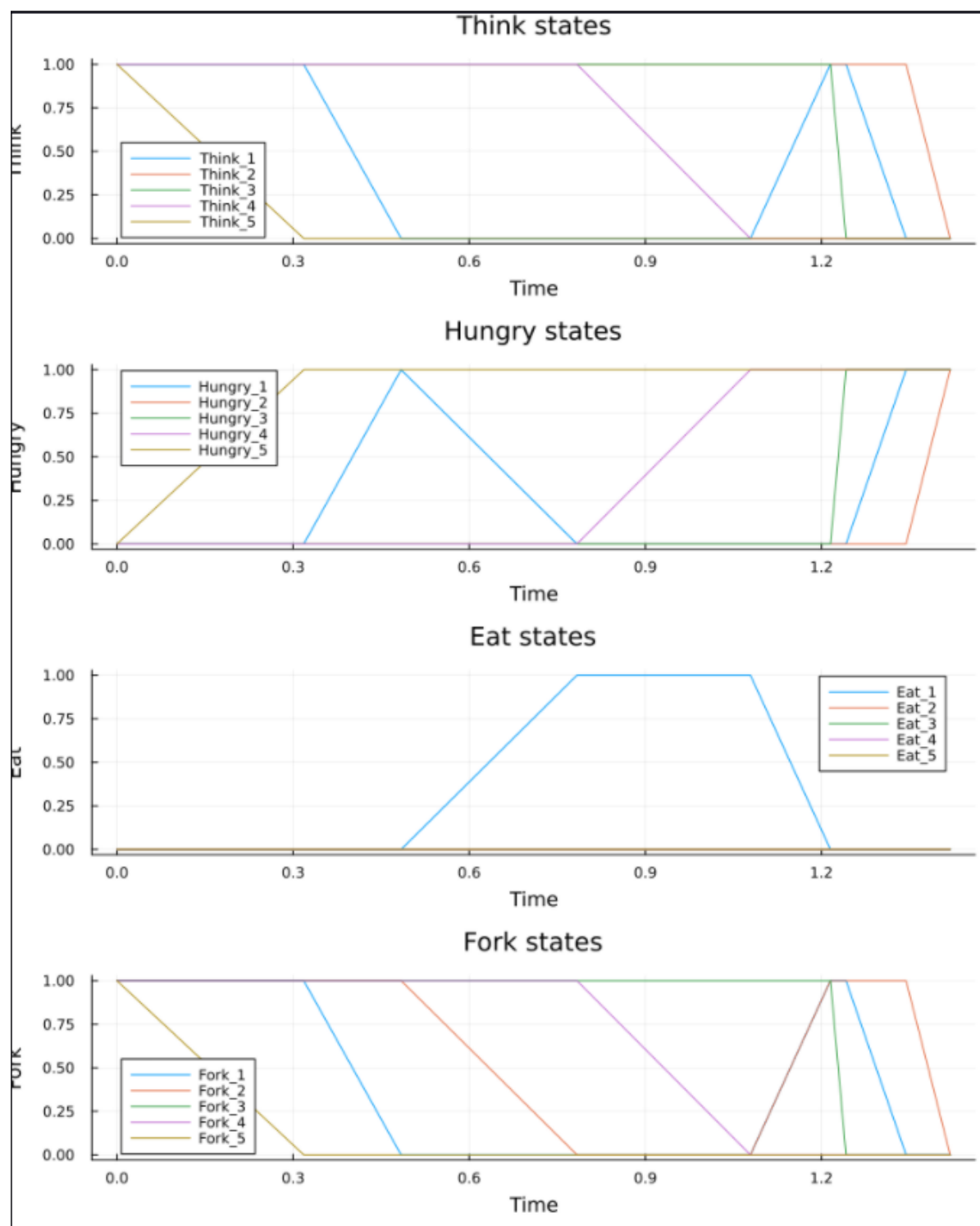


Рис. 3.5: Рис

2. В сети с арбитром наблюдается постоянное чередование приёма пищи. (рис.

3.6).

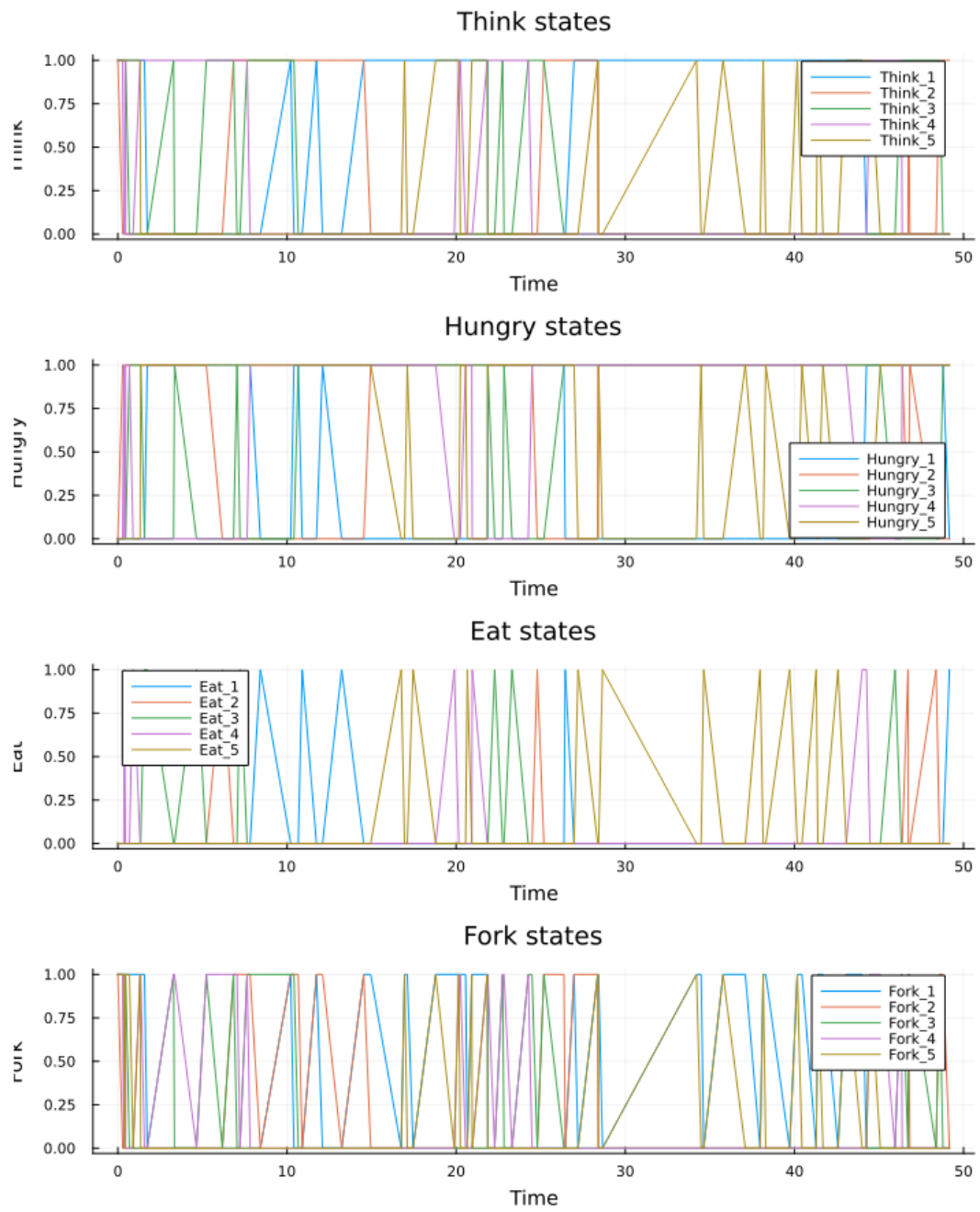


Рис. 3.6: Рис

3.3 Анимация процесса

- Для наглядной демонстрации динамики работы сети Петри во времени.
- Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели. (рис. 3.7).



```
using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u, legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = ${round(row.time, digits=2)}",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end

gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

Рис. 3.7: Рис

- Анимация делает абстрактные состояния сети Петри живыми.
- Видно, как фишки перемещаются между позициями: философы переходят из Think в Hungry, затем в Eat и обратно. (рис. 3.8).

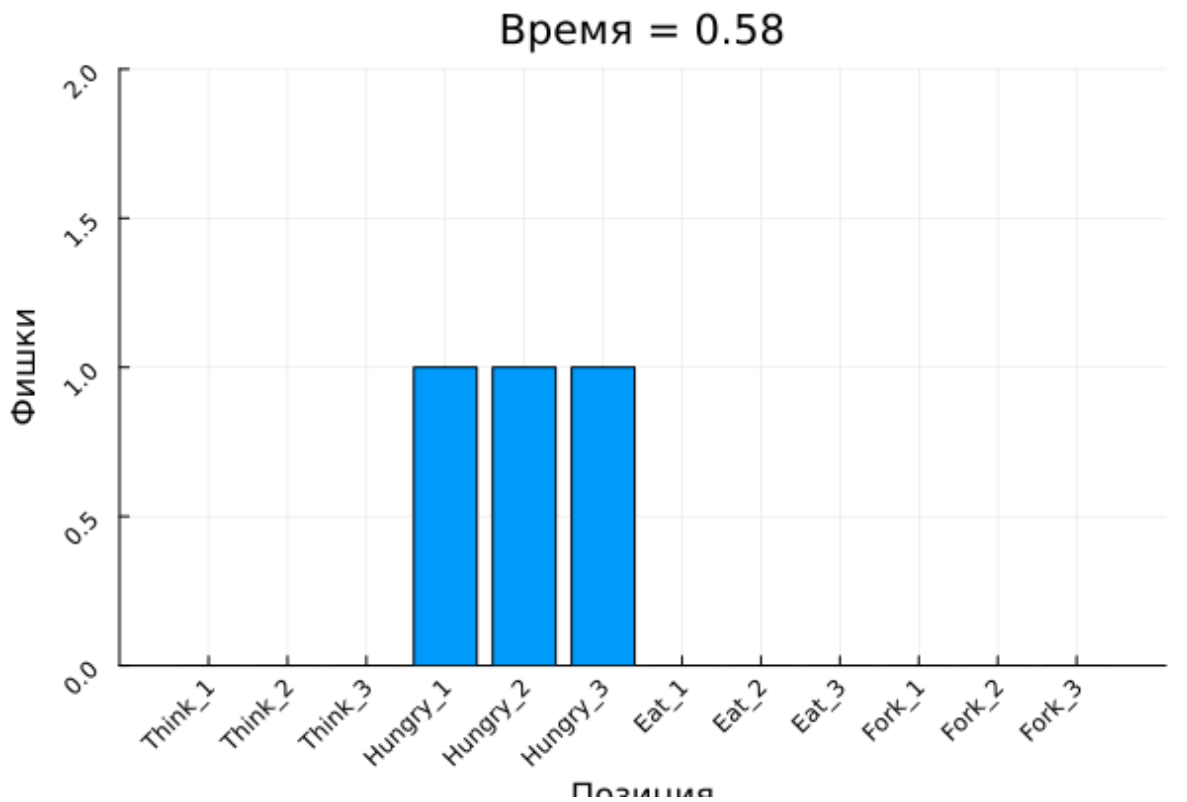


Рис. 3.8: Рис

3.4 Итоговый отчёт

- Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i).
- Скрипт генерирует сводный график. (рис. 3.9).

```
Ouvrir ▾
dining_philosophers_report.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/lab05/project/scripts
dining_philosophers_report.jl
DiningPhilosophers.jl
dining_philosophers.jl
dining_philosophers_animation.jl
dining_philosophers_report.jl

using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5

# Столбцы для состояния "Ест"
eat_cols = [Symbol("Eat_{$i}") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix{Float64}(df_classic[:, eat_cols]),
    label = ["φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix{Float64}(df_arbiter[:, eat_cols]),
    label = ["φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
)

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotdir("final_report.png"))

println("Отчёт сохранён в plots/final_report.png")
```

Рис. 3.9: Рис

1. Классическая сеть: на графике видно, что через некоторое время все линии Eat_i падают до нуля и остаются на нуле
 - Это и есть deadlock.
 - Философы больше не могут есть.
2. Сеть с арбитром: линии Eat_i колеблются, всегда есть хотя бы один философ, который ест (или они поочерёдно получают доступ).
 - Отсутствие длительных нулевых периодов подтверждает, что система жива и не блокируется. (рис. 3.10).

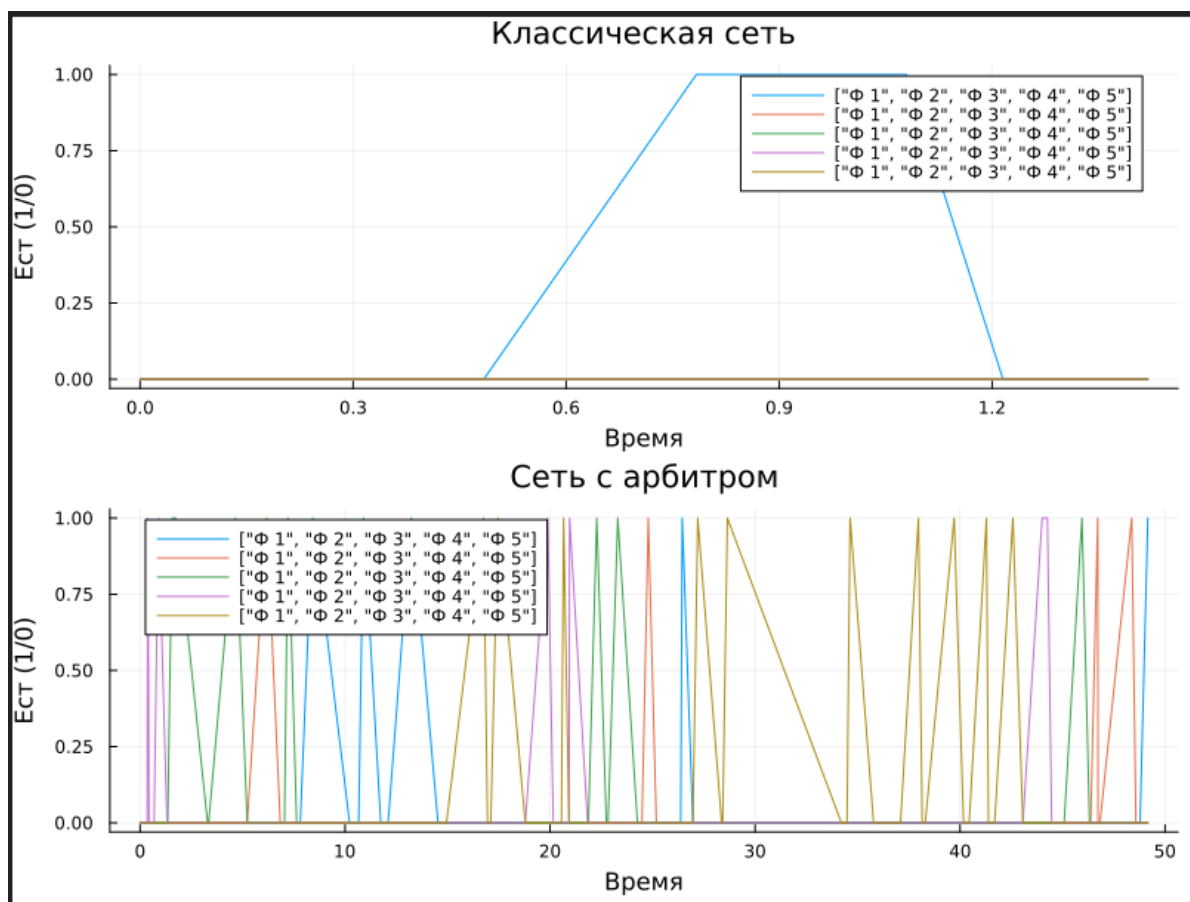


Рис. 3.10: Рис

4 Выводы

- Я выполнила лабораторную работу.

Список литературы